챕터 8. 책 뒤에 색인을 붙였다. Elasticsearch + Spring Data

이번 챕터가 끝나면

- RDB의 `LIKE '%강의%'` 가 10만 건에서 왜 5초를 먹는지, 왜 띄어쓰기·영문 혼용에 약한지를 체감합니다
- 도서관 색인 카드의 비유로 **역색인(Inverted Index)** 과 **분석기(Analyzer)** 의 동작을 한 장면으로 잡습니다
- Docker Compose에 Elasticsearch + Kibana를 얹고, Spring Boot에 `spring-boot-starter-data-elasticsearch` 를 끼워 `@Document` · `@Field` 로 인덱스를 매핑합니다
- **RDB 원본 저장 + ES 색인 동시 저장(dual write)** 과 **ES에서 id 찾고 RDB로 재조회**하는 검색 흐름을 컨트롤러 한 개와 서비스 한 개로 조립합니다
- `multi_match` 의 `title^3` 가중치와 `fuzziness("AUTO")` 로 오타 허용까지 Kibana Dev Tools와 Postman에서 직접 확인합니다
- 맨 끝에서 RDB와 ES 두 저장소의 **이중 저장이 부를 불일치 문제**를 살짝 건드려, 다음 챕터의 문을 엽니다

이번 챕터는 "도서관 사서가 10만 권의 책을 매번 뒤지는 대신 책 뒤에 색인 카드를 붙여 둔 이야기"입니다

챕터 7에서 동료는 "DB에 `LIKE '%강의%'` 를 걸었더니 10만 건에서 5초가 걸린다"고 말하고 자리를 떴습니다. 팀장은 한마디를 남겼습니다. "DB로 검색을 하면 그래요." 이 챕터에서는 그 5초를 ms 단위로 끊어 보겠습니다. RDB에는 계속 원본을 두되, 검색은 Elasticsearch라는 별도의 검색 엔진에 맡깁니다. 엔진은 문서를 저장할 때 분석기로 문장을 단어로 쪼개 두고, 그 단어마다 "이 단어가 들어 있는 문서 번호 목록"을 뒤쪽 색인 카드처럼 만들어 둡니다. 검색 요청이 오면 카드 한 장만 보고 번호 목록으로 바로 점프합니다. 여기에 오타까지 허용하는 Fuzzy를 얹어서, "갈럭시"라고 쳐도 "갤럭시" 가 걸리게 만듭니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 8도 Spring Boot 하나로 시작합니다. 챕터 7의 HLS 레포와 별개의 레포입니다. 이번 레포에는 Docker Compose 파일, Spring Boot 앱, `DeviceEntity` · `DeviceDocument` 한 쌍과 검색 컨트롤러 하나가 들어 있습니다.

레포	용도	주소
docker-elasticsearch	Docker Compose(ES + Kibana + Spring) + Spring Data Elasticsearch	`github.com/metacoding-11-spring-reference/docker-elasticsearch`

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/docker-elasticsearch.git
cd docker-elasticsearch
```

파일 구조는 이렇습니다.

docker-elasticsearch 디렉토리

```
docker-elasticsearch/
├── elasticsearch/
│   └── docker-compose.yml                                  # [실습] ES + Kibana + app
├── spring-elasticsearch/
│   ├── build.gradle                                        # [실습] spring-data-elasticsearch 의존성
│   └── src/main/
│       ├── java/com/metacoding/spring_elasticsearch/
│       │   ├── device/
│       │   │   ├── DeviceEntity.java                       # [실습] JPA 엔티티
│       │   │   ├── DeviceDocument.java                     # [실습] @Document · @Field
│       │   │   ├── DeviceJpaRepository.java                # [참고] JpaRepository
```

```
│       │   │   └── DeviceSearchRepository.java                  # [참고] Elast
icsearchRepository
│       │   └── ElasticSearch/
│       │       ├── ElasticSearchService.java                    # [실습] dua
l write · 검색
│       │       └── ElasticSearchSearchController.java           # [실습] POS
T /devices · GET /search
│       └── resources/
│           └── application.properties                           # [실습] ES UR
I
└── README.md                                                    # [참고] 샘플 J
SON
```

> **Elasticsearch는 Spring 안에 포함되지 않는 별도의 서버입니다.** Redis · S3 · RabbitMQ 와 마찬가지로, Spring 앱을 띄우기 전에 먼저 ES 프로세스가 떠 있어야 합니다. 이 실습은 Docker Compose 로 ES · Kibana · Spring 앱 세 컨테이너를 한 번에 올립니다.

2. 실습 환경 구축

터미널 실습 환경 확인

```
docker --version
docker compose version
```

Docker Desktop 이 떠 있어야 합니다. 이 챕터의 ES 는 `8.19.8` 이미지를 씁니다. JVM 힙을 1GB 로 잡기 때문에, 호스트 메모리 여유가 3GB 이상이면 편합니다.

> **왜 ES 를 로컬 브로커로 설치하지 않고 Docker 로 쓰나요**
>
> Elasticsearch 는 JVM 기반 서버라 설치 과정에 JDK · 설정 · 데몬 등록이 따라붙습니다. 버전이 엇갈리면 호스트 환경이 쉽게 오염됩니다. Docker 이미지 하나를 올리면 이 과정을 건너뛰고, 실습이 끝난 뒤 컨테이너만 내리면 원래 상태로 돌아갑니다. 실무에서도 개발 환경은 Docker, 스테이징·운영은 매니지드(Elastic Cloud · AWS OpenSearch) 로 분리하는 구성이 흔합니다.

3. 사용할 구성 요소

이번 챕터에 새로 얹히는 재료는 여섯 가지입니다.

재료	역할
`docker.elastic.co/elasticsearch/elasticsearch:8.19.8`	검색 서버 본체. `9200` 포트로 REST 요청을 받음
`docker.elastic.co/kibana/kibana:8.19.8`	ES 관리·디버그 UI. `5601` 포트. Dev Tools 콘솔로 직접 쿼리 가능
`spring-boot-starter-data-elasticsearch`	Spring Data 가 ES 클라이언트·Repository 추상화를 제공
`@Document(indexName = "...")`	자바 클래스를 ES 인덱스 문서로 매핑
`@Field(type = FieldType.Text)`	필드를 분석기로 토큰화되는 `text` 타입으로 지정
`NativeQuery` · `ElasticsearchOperations`	`multi_match` · `fuzziness` 같은 DSL 쿼리를 자바 코드로 조립

4. 실습 순서

이번 챕터는 이 순서로 흘러갑니다.

1. `8.1` . 동료의 "검색이 5초 걸려요"
2. `8.2` . DB 로 검색하면 왜 무너지는가 (Try/Fail)
3. `8.3` . 책 뒤에 색인 카드를 붙이는 도서관 (역색인의 비유)
4. `8.4` . Docker 로 ES + Kibana 띄우기
5. `8.5` . RDB 엔티티 + ES 문서 한 쌍 만들기 (`@Document` · `@Field`)
6. `8.6` . 동시 저장(dual write) 과 검색 흐름 (id 로 RDB 재조회)
7. `8.7` . 오타 허용 검색(Fuzzy) 과 AUTO 규칙
8. `8.8` . Kibana 와 Postman 으로 전체 검증
9. `8.9` . 두 저장소를 쓰는 이상 피할 수 없는 문제

1에서 문제를 듣고, 2에서 `LIKE` 의 한계를 직접 재현합니다. 3에서 도서관 비유로 역색인을 잡고, 4~6에서 ES 를 붙여 저장·검색을 한 조각씩 조립합니다. 7에서 Fuzzy 로 오타까지 잡고, 8에서 Kibana · Postman 으로 확인합니다. 9에서 "두 저장소를 쓴다"는 구조가 부를 새 문제를 살짝 엿보고, 다음 챕터의 문을 엽니다.

8.1 "검색이 5초 걸려요"

챕터 7을 닫은 다음 날 오전이었습니다. 오픈이는 커피를 들고 자리에 앉았는데, 동료가 노트북을 안고 쫓아왔습니다.

동료 : "오픈이 님, 이번엔 검색이에요."

오픈이 : "어떤 검색이요?"

동료 : "영상 제목으로 검색하는 창이요. 지금 DB에 `LIKE '%강의%'` 로 걸어 놨는데, 강의가 10만 건 넘으니까 5초씩 걸려요."

모니터 위에 Postman 창이 떠 있었습니다. `GET /search?keyword=강의` 를 누르자 오른쪽 아래 응답 시간 배지가 `5.12 s` 에서 멈춰 있었습니다. 오픈이는 손가락으로 테이블을 두드리면서 숫자를 곱해 봤습니다. 동시 접속 100 명, 검색 한 번에 5 초. 서버 커넥션 풀이 금방 바닥날 그림이었습니다.

인덱스가 안 탄다.

동료 : "그리고 더 문제는요, '강 의' 로 띄어 쓰거나 'Lecture' 로 치면 아예 안 잡혀요. 사용자는 검색창에 뭐라고 쳐도 다 나오길 바라잖아요."

오픈이 : "자바 기초 치면 기초 자바는요."

동료 : "안 나와요. 항의가 들어와요."

팀장이 뒤쪽에서 의자를 뒤로 빼면서 모니터 쪽을 흘깃 봤습니다.

팀장 : "도서관 사서가 매번 10만 권을 다 펼쳐서 읽으면요."

오픈이는 바로 대답이 안 나왔습니다.

팀장 : "책 뒤에 '찾아보기' 붙어 있잖아요. 그거 누가 만들어 놨죠."

오픈이 : "사서가 미리요."

팀장 : "그 '미리' 를 DB 가 해 줄까요."

팀장은 그 말만 남기고 모니터로 돌아갔습니다.

DB 는 원본 저장소다. 검색 엔진이 아니다.

오픈이는 수첩을 펼쳤습니다. 챕터 7 마지막 장에 "다음 챕터에서 Elasticsearch 를 얹습니다" 라고 적어 둔 줄이 그대로 남아 있었습니다. 그 아래에 오늘 받은 문제 세 줄을 옮겨 적었습니다.

```
— 문제 정리 —

1. 속도
   · LIKE '%강의%' 가 10만 건에서 5초
   · 앞뒤 와일드카드 때문에 DB 인덱스가 동작하지 않음

2. 단어 단위 검색
   · 강의 와 강 의, Lecture 가 다 다른 문자열로 취급됨
   · "자바 기초" 로 쳤을 때 "기초 자바" 는 안 나옴

3. 오타
   · 갈럭시 를 치면 갤럭시 가 아예 안 걸림
   · 사용자 입장에서는 "없는 것" 으로 보임
```

8.2 DB 로 검색하면 왜 무너지는가

오픈이는 먼저 "5초가 정말 DB 탓인가" 를 확인하기로 했습니다. 추측으로 엔진을 바꿀 수는 없었습니다.

IntelliJ 를 열고 H2 콘솔에 붙어서 `EXPLAIN` 을 찍어 봤습니다.

한 번 맞춰 보자.

[GEMINI PROMPT: 왼쪽에 DB 아이콘(원통형 실린더)과 오른쪽에 검색창 아이콘을 배치. 가운데에 10만 장의 책 종이가 쌓여 있고, 사서 한 명이 한 장 한 장 펼쳐 보는 장면. 사서의 얼굴에는 땀방울이 떨어지고 있음. 책 더미 위에 "LIKE '%강의%'" 라는 SQL 문이 떠 있음. 배경은 도서관, 갈색 톤. 상단에 큰 시계가 5초에서 멈춰 있음]

그림 8-1. `LIKE '%강의%'` 는 DB 에게 "10만 권을 다 열어 보라"고 시키는 명령입니다

쿼리는 이렇게 생겼습니다. 구현은 이야기 파트 범위 밖이고, 흐름만 보겠습니다. `device_entity` 테이블에서 `content` 컬럼에 `%강의%` 가 들어 있는 행을 모두 찾으라는 요청입니다. DB 는 `content` 컬럼에 B-Tree 인덱스가 걸려 있어도, **앞에 `%` 가 붙은 순간 인덱스의 시작 지점을 잡지 못합니다.** 인덱스는 "ㄱ 으로 시작하는 것", "ㄱㅏ 로 시작하는 것" 같은 **앞에서부터 정렬된 사전** 이기 때문입니다. "어디엔가 강의가 들어 있는 것"은 사전의 어느 페이지를 펴야 할지 알 수 없습니다. 결국 10만 행 전부를 한 번씩 읽습니다.

5초는 이 "전부 읽기" 의 결과였습니다.

두 번째 문제로 넘어갔습니다. 오픈이가 테이블에 직접 값을 세 건 넣어 놓고 검색해 봤습니다.

- "자바 기초 강의"
- "기초 자바 프로그래밍"
- "Java Basic Lecture"

사용자가 `자바 기초` 를 친 상황. `LIKE '%자바 기초%'` 는 첫 번째 행만 잡았습니다. 두 번째 행은 `기초 자바` 순서라 걸리지 않았습니다. 세 번째 행은 영어라 아예 후보도 아니었습니다. DB 에게 "자바" 와 "기초" 를 따로 찾아서 둘 다 들어간 행을 달라고 시키려면, `WHERE content LIKE '%자바%' AND content LIKE '%기초%'` 로 조건을 쪼개야 했습니다. 조건마다 풀스캔이 한 번씩 도는 셈이었습니다.

오타는 더 답이 없었습니다. `갈럭시` 라는 문자열 자체가 테이블 어디에도 저장돼 있지 않았기 때문에, LIKE 로는 "비슷한 것" 을 찾을 방법이 없었습니다. DB 에서 이 문제를 풀려면 **한 글자씩 바꿔 가면서 여러 번 쿼리를 날리는 로직** 을 애플리케이션 쪽에 직접 만들어야 했습니다. 10만 건 테이블에서 이걸 반복하는 건 비현실적이었습니다.

오픈이는 세 번째 줄 아래에 결론을 달았습니다.

DB 는 정확한 값·범위·조인에 강하지, "검색창" 요구에는 약하다.

동료 : "그래서 뭐로 바꿔요?"

오픈이 : "검색 엔진이요. 검색 엔진은 이 문제들을 풀려고 설계된 도구래요."

동료 : "그 검색 엔진이 DB 를 대신해요?"

오픈이 : "아니요. 원본은 DB 에 계속 두고, 검색용 복사본만 검색 엔진에 또 저장할 거예요. DB 는 안전금고, 엔진은 찾아보기. 역할을 분리해요."

8.3 책 뒤에 색인 카드를 붙이는 도서관

오픈이는 수첩 새 장을 펴고 그림을 그렸습니다.

책이 10만 권 꽂혀 있는 도서관에서, 누군가 "'워치' 들어간 책 가져와요" 라고 물었다고 했습니다. 사서가 책을 한 권씩 꺼내 본문을 넘겨 보면 10만 번의 넘김이 필요합니다. 책이 많아질수록 시간은 선형으로 늘어납니다. 대출대 앞이 막힙니다. 지금의 `LIKE '%워치%'` 가 이 장면입니다.

대신 도서관이 책을 들여올 때마다 **책 뒤 세 페이지에 단어 색인을 뽑아 붙여 둔다**고 해 봅니다. "갤럭시 워치" 라는 제목의 책이 들어오면 "갤럭시" 와 "워치" 를 카드에 적고, 색인 서랍의 `ㄱ` 칸과 `ㅇ` 칸에 각각 "책 번호 1" 을 기록합니다. 다음에 "갤럭시 S24" 가 들어오면 `ㄱ` 칸의 갤럭시 카드에 "책 번호 2" 를 추가로 적고, `ㅅ` 칸에 "S24: 2" 를 새로 만듭니다. "아이폰 워치" 가 들어오면 `ㅇ` 칸의 워치 카드에 "3" 을 붙이고, 새로 `ㅇ` 칸에 "아이폰: 3" 도 만듭니다.

[GEMINI PROMPT: 도서관 장면. 왼쪽에 책 3권이 꽂혀 있고 각 책 제목이 "갤럭시 워치", "갤럭시 S24", "아이폰 워치" 로 보임. 책 뒤표지에 색인 카드가 붙어 있는 장면. 오른쪽에는 목재 색인 서랍장이 있고, 각 서랍에 한글 자음(ㄱ, ㅇ, ㅅ) 라벨이 붙어 있음. ㄱ 서랍에서 "갤럭시 → 책1, 책2" 카드가 튀어 나와 있음. ㅇ 서랍에서 "워치 → 책1, 책3" 카드가 튀어 나와 있음. 사서는 손에 카드 한 장만 들고 서가 번호를 가리키는 모습. 갈색 · 베이지 톤, 따뜻한 조명]

그림 8-2. 책 뒤 색인을 미리 붙여 두면, 사서는 카드 한 장만 보고 서가 번호로 바로 점프합니다

이제 누군가 "워치" 를 물어보면 사서는 본문을 넘기지 않습니다. `ㅇ` 칸의 워치 카드 한 장을 꺼냅니다. 거기 "1, 3" 이라고 적혀 있습니다. 사서는 1번 서가와 3번 서가로 가서 책을 내 줍니다. 10만 권이 꽂혀 있어도, 카드 한 장만 보면 끝납니다.

팀장 : "그게 Elasticsearch 가 하는 일이에요."

오픈이는 수첩에 선을 그었습니다. 왼쪽에 "책 → 카드" 라 적고, 오른쪽에 "카드 → 책 번호 목록" 이라 적었습니다. 왼쪽은 원본 책의 자연스러운 방향, 오른쪽은 그것을 뒤집은 방향이었습니다. 그래서 이름이 역색인이라고 팀장이 한마디를 보탰습니다. 카드를 만드는 일은 책이 들어올 때 한 번만 합니다. 그 대가로 찾는 일은 10만 번 넘김에서 카드 한 장 읽기로 바뀝니다.

사서는 또 한 가지 일을 더 합니다. 책 제목 "갤럭시 워치" 를 그대로 한 장의 카드에 쓰지 않습니다. 띄어쓰기를 따라 **"갤럭시"** 와 **"워치"** 두 장으로 쪼갭니다. 이 작업을 **분석(analyze)** 이라고 부릅니다. 분석기가 문장을 단어(토큰) 로 쪼개는 규칙을 알고 있습니다. 한국어는 조사·어미가 붙는 언어라 단순 공백 분리로는 한계가 있고, `nori` 같은 한국어 분석기를 쓰면 "연차 유급 휴가" 를 "연차 / 유급 / 휴가" 로 더 영리하게 자릅니다.

검색 때도 같은 분석기가 한 번 더 돕니다. 사용자가 `자바 기초` 를 치면, 분석기가 먼저 `자바` 와 `기초` 두 토큰으로 자르고, 각 토큰의 카드를 서랍에서 꺼냅니다. 두 카드의 교집합이 "자바 와 기초 둘 다 들어간 책 번호" 입니다. 순서는 상관없습니다. "자바 기초" 와 "기초 자바" 둘 다 같은 두 토큰으로 쪼개지기 때문입니다. 첫 문제(순서에 민감한 LIKE) 가 여기서 풀립니다.

영어도 풀립니다. 문서 저장 시 `Java Basic Lecture` 는 `java`, `basic`, `lecture` 세 토큰으로 쪼개지고, 검색어 `Java` 도 `java` 로 정규화됩니다. 대소문자·띄어쓰기 차이가 분석 단계에서 정리됩니다.

오타는 한 단계 더 나간 트릭이 필요합니다. 사용자가 `갈럭시` 를 쳤는데 서랍에는 `갤럭시` 카드밖에 없습니다. 검색 엔진은 "글자 한 번 바꾸면 같아지는 카드 있나요" 라고 한 번 더 둘러봅니다. 이게 **Fuzzy** 입니다. 얼마나 많이 바꿔도 되는지는 단어 길이에 따라 자동으로 정합니다 (`AUTO`).

수첩 옆장에 오픈이는 세 단어를 적었습니다.

```
— 기억할 것 —

1. 역색인 (Inverted Index)
   · "단어 → 문서 번호 목록" 으로 뒤집어 둔 구조
   · 문서를 저장할 때 한 번 만들고, 검색 시에는 카드 한 장만 봄

2. 분석기 (Analyzer)
   · 문장을 토큰(단어) 으로 쪼개는 규칙
   · 저장·검색 시 같은 분석기가 한 번씩 동작해야 순서·대소문자·띄어쓰기 차이가 흡수됨

3. Fuzzy
   · 편집 거리(한 글자 교체·추가·삭제) 안의 이웃 토큰까지 후보로 끌어옴
   · AUTO 는 단어 길이에 따라 허용 거리를 자동 결정
```

동료 : "그럼 DB 는 안 써요."

오픈이 : "써요. 원본은 DB 에 둬요. 수정도 DB 에서 해요. 검색만 엔진한테 시키는 거예요. 검색 엔진이 알려 준 번호로 DB 에서 다시 꺼내요."

동료 : "왜 두 번 저장해요."

오픈이 : "DB 는 금고, 엔진은 찾아보기. 금고가 없으면 원본이 날아가고, 찾아보기 없으면 10만 번을 뒤져야 돼요. 둘 다 필요해요."

동료가 고개를 끄덕였습니다. 오픈이는 이제 실제로 엔진을 하나 띄우러 갔습니다.

이제 직접 만들어 보겠습니다.

8.4 Docker 로 Elasticsearch 를 띄운다

Elasticsearch 는 Spring 내부 라이브러리가 아니라 **별도의 서버** 입니다. Redis 가 그랬듯이, 먼저 서버가 떠 있어야 Spring 이 접속할 수 있습니다. 이번 레포의 `elasticsearch/docker-compose.yml` 은 세 컨테이너를 한 번에 띄웁니다. ES 본체(9200 포트), Kibana(5601 포트, 관리 UI), Spring 앱(8080 포트) 입니다.

그림 8-3. Docker Compose 가 띄우는 세 컨테이너

Elasticsearch	검색 서버 · 포트 9200 · REST API · 단일 노드 · 힙 1GB
Kibana	관리·디버그 UI · 포트 5601 · Dev Tools 콘솔에서 DSL 직접 실행
Spring 앱	실습 API · 포트 8080 · 같은 es-network 에서 `elasticsearch:9200` 으로 접속

세 컨테이너는 `es-network` 하나로 묶여 있습니다

`elasticsearch/docker-compose.yml` 을 엽니다. 이 파일은 레포에 이미 작성돼 있고, 수정할 필요는 없습니다. 구조만 읽어 보겠습니다.

설명 1 elasticsearch/docker-compose.yml. ES + Kibana + Spring 세 컨테이너

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.19.8
    container_name: elasticsearch
    ports:
      - "9200:9200"
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
    networks:
      - es-network

  kibana:
```

```
    image: docker.elastic.co/kibana/kibana:8.19.8
    ports:
      - "5601:5601"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    networks:
      - es-network

  app:
    build:
      context: .
      dockerfile: elasticsearch/Dockerfile
    container_name: spring-elasticsearch-app
    ports:
      - "8080:8080"
    networks:
      - es-network

networks:
  es-network:
    driver: bridge
```

세 가지만 기억하면 됩니다.

설정	역할
`discovery.type=single-node`	클러스터가 아닌 단일 노드로 띄움. 로컬 실습용
`xpack.security.enabled=false`	인증·TLS 끔. 로컬 테스트에서만 허용
`ES_JAVA_OPTS=-Xms1g -Xmx1g`	JVM 힙 1GB 고정. 호스트 메모리 보호

`ELASTICSEARCH_HOSTS=http://elasticsearch:9200` 에서 `elasticsearch` 는 호스트 이름이 아니라 **같은 Compose 네트워크의 서비스 이름** 입니다. Kibana 컨테이너가 같은 `es-network` 에 있기 때문에 이 이름으로 ES 를 찾습니다.

Spring 앱이 같은 Compose 안에 있을 때 접속 주소와 로컬에서 개발할 때 접속 주소가 다른 점도 짚어 둡니다. 이 구분은 실습 내내 한 번은 만나게 됩니다.

그림 8-4. 같은 9200 포트를 두 이름으로 부르는 이유



접속 주소: `http://localhost:9200`
Docker 가 호스트의 9200 포트로 ES 컨테이너를 노출해 두기 때문

Docker 컨테이너 실행

접속 주소: `http://elasticsearch:9200`
같은 `es-network` 안에서 서비스 이름으로 찾기 때문

한 포트, 두 이름. 실행 위치에 따라 갈립니다

`spring-elasticsearch/src/main/resources/application.properties` 를 열면 Spring 이 어느 쪽으로 접속할지 한 줄로 정해져 있습니다.

실습 1 spring-elasticsearch/src/main/resources/application.properties. ES URI 설정

```
spring.datasource.url=jdbc:h2:mem:testdb;MODE=MySQL
spring.datasource.username=sa
spring.datasource.password=

# TODO: Spring 을 Docker 로 돌리면 elasticsearch:9200, 로컬로 돌리면 localhost:9200
spring.elasticsearch.uris=http://elasticsearch:9200
```

이 실습은 `docker compose up -d` 로 Spring 까지 함께 띄우기 때문에 `elasticsearch:9200` 으로 둡니다. 로컬에서 IntelliJ 로 Spring 만 따로 돌리면 `localhost:9200` 으로 바꾸고 ES · Kibana 두 컨테이너만 띄우면 됩니다.

`spring-elasticsearch/build.gradle` 에는 이 한 줄이 들어가 있습니다.

실습 2 spring-elasticsearch/build.gradle. 의존성 추가

```
dependencies {
    // TODO: Spring Data Elasticsearch 스타터 추가
    implementation 'org.springframework.boot:spring-boot-starter-data-elasticsearch'
    // ... 나머지 의존성
}
```

이 스타터가 들어오면 Spring 이 `spring.elasticsearch.uris` 값을 읽어 `ElasticsearchClient` · `ElasticsearchOperations` 빈을 자동으로 등록합니다. 우리는 이 빈을 서비스에서 주입받아 쓰기만 하면 됩니다.

레포 루트에서 한 줄을 실행합니다.

터미널 실험 8-1 실행. ES · Kibana · Spring 세 컨테이너 띄우기

```
docker compose -f elasticsearch/docker-compose.yml up -d
```

`-d` 옵션으로 백그라운드에서 띄웁니다. ES 는 부팅에 30초 정도 걸립니다. `docker logs -f elasticsearch` 로 로그를 지켜보면 `started` 라인이 뜰 때 준비 완료입니다.

확인은 두 가지로 합니다.

터미널 실험 8-2 실행. ES · Kibana 응답 확인

```
curl http://localhost:9200
open http://localhost:5601
```

[CAPTURE NEEDED: assets/CH08/terminal/08_es-health.png | 터미널에서 curl http://localhost:9200 실행 결과. JSON 응답에 name, cluster_name, cluster_uuid, version.number: "8.19.8", tagline: "You Know, for Search" 가 찍혀 있음]

그림 8-5. ES 가 `You Know, for Search` 한 줄을 돌려주면 서버는 정상입니다

Kibana 는 브라우저에서 `http://localhost:5601` 로 접속합니다. 첫 로드에 10초쯤 걸립니다. 좌측 메뉴에서 `Management` → `Dev Tools` 로 들어가면 ES 에 직접 쿼리를 날릴 수 있는 콘솔이 열립니다. 이 콘솔은 이후 실습에서 인덱스·문서를 눈으로 확인할 때 계속 씁니다.

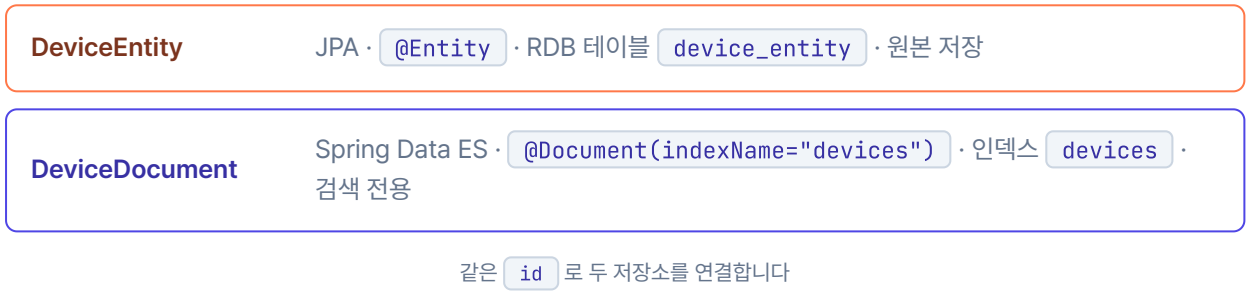
ES 가 안 뜰 때 점검 순서

- **메모리 부족**: `docker logs elasticsearch` 에 `out of memory` 가 찍히면 힙(`ES_JAVA_OPTS`) 을 512m 으로 줄이거나 호스트 여유 메모리를 확보합니다
- **포트 충돌**: 9200 포트가 이미 쓰이면 `docker ps` 로 다른 컨테이너를 내리거나 compose 의 포트 매핑을 `9201:9200` 으로 바꿉니다
- **Kibana 만 빨간불**: ES 보다 먼저 떠서 `Kibana server is not ready yet` 이 나오면 `docker restart kibana` 한 번으로 복구됩니다
- **보안**: 운영에서는 절대 `xpack.security.enabled=false` 를 쓰지 않습니다. 로컬 실습 한정

8.5 엔티티 한 개와 문서 한 개

ES 가 올라왔으니 이제 Spring 쪽 모델을 만듭니다. 원본은 RDB, 색인은 ES. 같은 데이터를 두 저장소에 각각 저장하기 때문에 **클래스도 두 개** 입니다. `DeviceEntity` 는 JPA 엔티티, `DeviceDocument` 는 ES 문서입니다. 두 클래스는 같은 필드를 공유하고, **같은 `id` 값을 쓰도록 맞춥니다**. 이유는 다음 절에서 나옵니다.

그림 8-6. 한 쌍의 클래스, 두 저장소



`spring-elasticsearch/src/main/java/com/metacoding/spring_elasticsearch/device/DeviceEntity.ja` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 3 device/DeviceEntity.java. RDB 원본 엔티티

```
@Entity
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DeviceEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // 1. 제목 필드. RDB 에도 저장되고 ES 에도 복사됨
    private String title;

    // 2. 본문 필드. 긴 설명 텍스트
```

```
    private String content;
}
```

이쪽은 평범한 JPA 엔티티입니다. 테이블명은 기본 전략으로 `device_entity` 가 됩니다. `IDENTITY` 전략을 쓰면 DB 가 `id` 를 자동 채번해 주고, Spring 은 `save()` 후 그 값을 돌려받을 수 있습니다. 이 `id` 가 뒤에서 ES 문서의 식별자로 그대로 재사용됩니다.

이어서 `spring-elasticsearch/src/main/java/com/metacoding/spring_elasticsearch/device/DeviceDocument.` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 4 device/DeviceDocument.java. ES 인덱스 문서

```
@Document(indexName = "devices")
@Getter
@NoArgsConstructor
@AllArgsConstructor
public class DeviceDocument {

    @Id
    private Long id;  // 1. RDB PK 와 동일한 값을 ES 문서 ID 로 사용

    // 2. Text 타입 = 분석기가 토큰으로 쪼개 역색인을 만듦
    @Field(type = FieldType.Text)
    private String title;

    @Field(type = FieldType.Text)
    private String content;
}
```

세 어노테이션이 하는 일은 각각 이렇습니다.

어노테이션	역할
`@Document(indexName = "devices")`	이 클래스가 `devices` 인덱스에 저장될 문서임을 선언 (RDB 의 테이블 지정에 해당)
`@Id` (Spring Data)	이 필드를 ES 문서의 `_id` 로 사용. JPA `@Id` 와 다른 클래스지만 역할은 동등
`@Field(type = FieldType.Text)`	이 필드는 분석기로 토큰화되는 `text` 타입. 역색인이 이 필드에 만들어짐

`FieldType` 에는 `Keyword` 도 있습니다. `Keyword` 는 **분석기를 거치지 않고 문자열 통째로** 저장합니다. 이메일·모델번호·태그처럼 "자르면 안 되는 값" 에 씁니다. 이번 실습의 제목·본문은 문장이고 **단어 단위로 쪼개야 검색이 되는** 필드이므로 둘 다 `Text` 로 둡니다.

> **역색인 (Inverted Index)**
>
> "단어 → 문서 번호 목록" 으로 뒤집은 검색용 자료구조. 문서를 저장할 때 분석기가 먼저 본문을 토큰으로 쪼개고, 각 토큰에 대해 "이 토큰이 들어 있는 문서들의 `_id` 목록" 을 작성·갱신한다. 검색 시에는 이 목록에서 바로 문서 번호들을 꺼내므로, 전체 문서를 스캔하지 않는다.
>
> **분석기 (Analyzer)**
>
> 텍스트를 토큰으로 쪼개는 규칙. `character filter` → `tokenizer` → `token filter` 세 단계로 동작한다. 기본 분석기는 공백·문장부호로 자르고, 한국어에는 `nori` 같은 형태소 분석기가 별도로 필요하다. 저장 시점과 검색 시점에 **같은 분석기** 가 적용돼야 한다.

두 클래스가 준비됐으니, Spring Data 의 Repository 도 두 개 필요합니다. 이 두 개는 `[참고]` 파일로 이미 작성돼 있습니다.

참고 device/DeviceJpaRepository.java · DeviceSearchRepository.java

```java
// RDB 저장·조회
public interface DeviceJpaRepository
        extends JpaRepository<DeviceEntity, Long> { }

// ES 저장·조회
public interface DeviceSearchRepository
        extends ElasticsearchRepository<DeviceDocument, Long> { }
```

Repository	역할	주요 메서드
`DeviceJpaRepository`	RDB 원본의 기본 CRUD	`save()`, `findById()`, `findAllById()`
`DeviceSearchRepository`	ES 문서의 기본 CRUD + 간단 검색	`save()`, `findById()`, 필드 기반 파생 쿼리

복잡한 검색 쿼리(`multi_match`, `fuzziness`, 가중치) 는 `DeviceSearchRepository` 로는 표현이 어렵기 때문에, 다음 절에서 `ElasticsearchOperations` 를 함께 씁니다.

8.6 저장은 둘, 검색은 하나

저장과 검색의 흐름을 한 장에 먼저 그립니다. 저장은 "RDB 에 먼저, ES 는 그다음" 순서. 검색은 "ES 에서 id 만 받아, RDB 로 재조회" 순서입니다.

그림 8-7. 저장 시퀀스입니다. RDB 가 먼저 PK 를 정하고, 같은 PK 로 ES 문서를 씁니다

그림 8-8. 검색 시퀀스입니다. ES 는 번호만 돌려주고, 본문은 RDB 가 돌려줍니다

이제 서비스 한 개에 두 흐름을 모두 씁니다. `spring-elasticsearch/src/main/java/com/metacoding/spring_elasticsearch/ElasticSearch/ElasticS` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 5 ElasticSearch/ElasticSearchService.java. 저장 흐름 (dual write)

```
@Service
@RequiredArgsConstructor
public class ElasticSearchService {

    private final DeviceJpaRepository deviceJpaRepository;
    private final DeviceSearchRepository deviceSearchRepository;
    private final ElasticsearchOperations operations;

    @Transactional
    public DeviceDocument saveDevices(DeviceDocument doc) {
        // TODO: RDB 에 먼저 저장하여 id 확보
        // 1. DTO → Entity 변환
        DeviceEntity entity = DeviceEntity.builder()
                .title(doc.getTitle())
                .content(doc.getContent())
                .build();

        // 2. RDB 저장. 돌아오는 saved.getId() 가 자동 채번된 PK
        DeviceEntity saved = deviceJpaRepository.save(entity);

        // 3. 같은 id 로 ES 문서 생성
        DeviceDocument savedDoc = new DeviceDocument(
                saved.getId(),
```

```
                saved.getTitle(),
                saved.getContent());

        // 4. ES 에 색인(인덱싱) 저장
        deviceSearchRepository.save(savedDoc);

        return savedDoc;
    }
}
```

순서가 중요합니다. `IDENTITY` 전략은 `save()` 가 끝나야 `id` 가 확정됩니다. ES 문서의 식별자로 같은 `id` 를 쓰려면 RDB 저장을 먼저 완료해야 합니다. 그래서 `@Transactional` 안에서도 `deviceJpaRepository.save()` 를 선행 호출합니다.

주입 빈	출처	쓰임
`deviceJpaRepository`	`spring-boot-starter-data-jpa`	RDB 저장·재조회
`deviceSearchRepository`	`spring-boot-starter-data-elasticsearch`	ES 기본 저장 (간단 CRUD)
`operations` (`ElasticsearchOperations`)	`spring-boot-starter-data-elasticsearch`	복잡한 DSL 쿼리 (`NativeQuery`) 실행

같은 서비스에 검색 메서드를 이어서 추가합니다.

실습 6 ElasticSearch/ElasticSearchService.java. 검색 흐름 (ES → RDB 재조회)

```
public List<DeviceEntity> searchAll(String keyword) {
    // TODO: multi_match + fuzziness 로 DSL 쿼리 조립
    // 1. title 에 가중치 3, content 에 가중치 1, fuzziness AUTO
    NativeQuery query = NativeQuery.builder()
            .withQuery(q -> q.bool(b -> b
                    .should(s -> s.multiMatch(m -> m
                            .fields("title^3", "content")
                            .query(keyword)
                            .fuzziness("AUTO")))
                    .minimumShouldMatch("1")))
            .build();

    // 2. ES 에 쿼리 전송. 응답은 SearchHits<DeviceDocument>
    var deviceHits = operations.search(query, DeviceDocument.class);
```

```
    // 3. 검색 결과에서 id 만 추출
    List<Long> ids = deviceHits.stream()
            .map(hit -> hit.getContent().getId())
            .toList();

    // 4. 같은 id 로 RDB 재조회. 최신 본문을 돌려줌
    return deviceJpaRepository.findAllById(ids);
}
```

`NativeQuery` 는 ES 의 Query DSL 을 자바 메서드 체이닝으로 표현합니다. 핵심 조각은 네 개입니다.

DSL 조각	역할
`multiMatch.fields("title^3", "content")`	두 필드를 동시에 검색. `title^3` 은 제목 매칭 점수를 3배로 키움
`.query(keyword)`	사용자가 입력한 검색어. 같은 분석기로 한 번 더 토큰화됨
`.fuzziness("AUTO")`	편집 거리 기반 오타 허용. 단어 길이별 허용량 자동 결정
`bool.should + minimumShouldMatch("1")`	"이 조건 중 최소 하나만 맞으면 포함" 이라는 느슨한 매칭

검색 결과에서 **본문을 쓰지 않고** `id` **만 뽑는** 이유가 세 번째 줄에 있습니다. ES 문서는 "검색용 복사본" 이기 때문에 시간이 지나면 RDB 원본과 살짝 어긋날 수 있습니다. 응답에 내보낼 데이터는 **언제나 원본인 RDB 에서 꺼내서** 신선함을 보장합니다. 이 한 줄이 "금고(RDB) vs 찾아보기(ES)" 원칙의 실제 구현입니다.

이제 컨트롤러 한 개로 두 엔드포인트를 묶습니다. `spring-elasticsearch/src/main/java/com/metacoding/spring_elasticsearch/ElasticSearch/ElasticS` 를 열고 TODO 의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 7 ElasticSearch/ElasticSearchSearchController.java. 저장·검색 API

```
@RestController
@RequiredArgsConstructor
public class ElasticSearchSearchController {

    private final ElasticSearchService elasticSearchService;
```

```
    // 1. 단건 저장. body 로 title · content 를 받음
    @PostMapping("/device")
    public DeviceDocument saveDevice(@RequestBody DeviceDocument doc) {
        return elasticSearchService.saveDevices(doc);
    }

    // 2. 여러 건 저장. 샘플 데이터 주입용
    @PostMapping("/devices")
    public List<DeviceDocument> saveDevices(@RequestBody List<DeviceDocument> docs) {
        return elasticSearchService.saveDeviceList(docs);
    }

    // 3. 검색. /search?keyword=갤럭시
    @GetMapping("/search")
    public List<DeviceEntity> search(@RequestParam("keyword") String keyword) {
        return elasticSearchService.searchAll(keyword);
    }
}
```

`saveDeviceList` 는 단건 `saveDevices` 를 내부에서 반복 호출하는 메서드입니다. 구현은 챕터 범위 밖(`rag-end` 참고) 이지만, 한 번에 10건 샘플을 넣기 위해 필요합니다.

Spring 을 다시 빌드하고 컨테이너를 재시작합니다.

터미널 실험 8-3 실행. Spring 컨테이너 재빌드

```
docker compose -f elasticsearch/docker-compose.yml up -d --build app
```

`--build` 옵션으로 `app` 컨테이너만 다시 빌드합니다. ES · Kibana 는 이미 떠 있으니 건드리지 않습니다. 빌드가 끝나면 `http://localhost:8080/search?keyword=test` 로 빈 응답(`[]`) 이 돌아와야 합니다. 아직 데이터를 안 넣었으니 빈 리스트가 정상입니다.

8.7 오타까지 허용하는 검색

`fuzziness("AUTO")` 한 줄이 무슨 일을 하는지 좀 더 파 봅니다. ES 는 검색어와 문서 토큰을 **편집 거리(Edit Distance)** 로 비교합니다. 편집 거리는 한 단어를 다른 단어로 만드는 최소 수정 횟수이고, 수정 방법은 세 가지입니다.

그림 8-9. 편집 거리 3종

교체 (Replace)	갤럭시 → 갈럭시 (갤 → 갈) · 편집 거리 1
추가 (Insert)	아이폰 → 아이폰X (X 삽입) · 편집 거리 1
삭제 (Delete)	갤럭시S → 갤럭시 (S 제거) · 편집 거리 1

거리가 작을수록 더 비슷한 단어로 판정됩니다

`AUTO` 는 검색어 길이에 따라 허용 거리를 자동으로 정합니다. 짧은 단어는 한 글자만 달라져도 의미가 크게 바뀌기 때문에 보수적으로 동작합니다.

검색어 길이	허용 편집 거리	예시
0~2	0 (오타 허용 안 함)	`TV` 는 `TB` 로 오타가 나도 매칭 안 됨
3~5	1	`갤럭시` ↔ `갈럭시` (한 글자 교체) 매칭
6+	2	`갤럭시워치` ↔ `갈럭시워츠` (두 글자 교체) 매칭

검색어가 분석기를 한 번 더 거친다는 점이 중요합니다. 사용자가 `갈럭시 워치` 를 치면, 분석기가 먼저 `갈럭시` 와 `워치` 두 토큰으로 쪼갭니다. `갈럭시` 는 길이 3 이라 허용 거리 1, `워치` 는 길이 2 라 허용 거리 0 이 적용됩니다. `갈럭시` 토큰은 `갤럭시` 카드와 매칭되지만, `워치` 는 정확히 일치해야만 매칭됩니다. 짧은 토큰의 오탐(false positive) 이 이 방식으로 걸러집니다.

Fuzzy 를 모든 곳에 쓰면 안 되는 이유

- **짧은 검색어**: 1~2 글자에 Fuzzy 를 켜면 `AB` 가 `AC` · `AD` · `BC` · `DB` 까지 번집니다. 오탐이 폭발합니다
- **정확 일치 필드**: SKU · 모델번호 · 이메일 같은 값은 `Keyword` 타입으로 저장하고 Fuzzy 를 끕니다. "SM-S928N" 이 "SM-S927N" 으로 잡히면 안 됩니다

- **필드 범위**: 제목에만 Fuzzy, 본문은 정확 매칭처럼 **필드별로 규칙을 달리** 할 수 있습니다

이번 실습은 `title` · `content` 둘 다에 `AUTO` 를 걸어 두고 동작을 체감하는 수준으로 둡니다. 실무에서 튜닝은 요구사항 · 도메인 · 비용을 보며 좁혀 갑니다.

8.8 Kibana 와 Postman 으로 증명

코드가 다 들어왔으니 실제로 검색이 되는지 확인할 차례입니다. 두 도구를 씁니다.

- **Kibana Dev Tools**: ES 에 직접 쿼리를 날려 "데이터가 제대로 색인됐는지" 확인
- **Postman**: Spring API 를 호출해 "Spring 까지 포함한 전체 흐름" 이 동작하는지 확인

8.8.1 샘플 데이터 10건 넣기

레포의 `README.md` 에 샘플 JSON 이 준비돼 있습니다. 10개의 디바이스(갤럭시 · 아이폰 · TV · 워치 · 스마트폰 등) 가 담겨 있습니다. Postman 에서 이 배열을 `POST /devices` 로 보냅니다.

터미널 실험 8-4 실행. 샘플 10건 일괄 저장

```
curl -X POST http://localhost:8080/devices \
  -H "Content-Type: application/json" \
  -d @sample-devices.json
```

[CAPTURE NEEDED: assets/CH08/terminal/08_sample-post.png | Postman POST http://localhost:8080/devices 요청. Body 에 10개 디바이스 JSON 배열. 응답은 200 OK 에 id 1~10 이 채워진 DeviceDocument 배열. 응답 시간은 수백 ms 대]

그림 8-10. 10건이 한 번에 RDB + ES 양쪽에 저장됩니다. 응답 배열의 `id` 는 자동 채번된 값입니다

H2 콘솔(`http://localhost:8080/h2-console`) 에서 `SELECT * FROM device_entity` 를 돌리면 10행이 보여야 합니다. 반대로 Kibana Dev Tools 에 다음 한 줄을 쳐서 ES 쪽도 확인합니다.

설명 2 Kibana Dev Tools 에서 인덱스 조회

```
GET /devices/_search
{
  "query": { "match_all": {} }
}
```

[CAPTURE NEEDED: assets/CH08/terminal/08_kibana-matchall.png | Kibana Dev Tools 우측 응답 창. hits.total.value: 10. hits.hits 배열에 id 1~10 의 DeviceDocument 가 _source 필드로 담겨 있음. _index: "devices", _score: 1.0]

그림 8-11. `match_all` 은 색인의 모든 문서를 돌려줍니다. `hits.total.value: 10` 이 나오면 양쪽 저장이 성공한 것입니다

8.8.2 `multi_match` 로 관련도 검색

이제 관련도 검색을 걸어 봅니다. Kibana Dev Tools 에서 먼저 ES 에게 직접 물어봅니다.

설명 3 기본 multi_match 쿼리. Kibana Dev Tools

```
GET /devices/_search
{
  "query": {
    "multi_match": {
      "query": "갤럭시",
      "fields": ["title^3", "content"]
    }
  }
}
```

[CAPTURE NEEDED: assets/CH08/terminal/08_kibana-match.png | Kibana Dev Tools. 응답에 hits.total.value: 3 정도. 상위 hit 의 _score 가 10점대. _source.title 에 "갤럭시 S24" 등 갤럭시가 들어간 문서. 아래쪽에 content 에만 갤럭시가 언급된 문서는 점수가 낮게 잡힘]

그림 8-12. `title^3` 덕분에 제목에 갤럭시가 들어간 문서가 본문에만 들어간 문서보다 위에 옵니다

점수가 왜 이렇게 나오는지는 Lucene 의 BM25 알고리즘이 결정하는데, 이 챕터 범위를 넘어갑니다. 지금은 "제목 매칭이 본문 매칭보다 3배 가중" 이 `^3` 의 효과라는 것만 짚고 넘어갑니다.

이어서 Spring 을 통해 똑같은 검색을 날립니다.

터미널 실험 8-5 실행. Spring 검색 API 호출

```
curl "http://localhost:8080/search?keyword=갤럭시"
```

[CAPTURE NEEDED: assets/CH08/terminal/08_spring-search.png | 터미널 curl 응답. JSON 배열에 DeviceEntity 3개. id 와 title 에 갤럭시가 포함된 엔티티들. 응답 시간은 수십 ms]

그림 8-13. Spring 이 ES 에서 `id` 를 받아 RDB 로 재조회한 결과입니다. 5초였던 검색이 수십 ms 로 떨어졌습니다

8.8.3 Fuzzy 로 오타까지

마지막으로 오타 허용을 확인합니다. Kibana 에서 먼저 봅니다.

설명 4 Fuzzy 쿼리. 갈럭시 → 갤럭시

```
GET /devices/_search
{
  "query": {
    "multi_match": {
      "query": "갈럭시",
      "fields": ["title^3", "content"],
      "fuzziness": "AUTO"
    }
  }
}
```

[CAPTURE NEEDED: assets/CH08/terminal/08_kibana-fuzzy.png | Kibana Dev Tools. 검색어 "갈럭시" 인데 응답 hits 에 title "갤럭시 S24", "갤럭시 워치" 등이 포함돼 있음. _score 는 정상 검색보다 낮게 나옴]

그림 8-14. `갈럭시` 를 치면 `갤럭시` 문서가 매칭됩니다. 편집 거리 1 안쪽이므로 `AUTO` 가 허용합니다

Spring API 로도 같은 오타를 보냅니다.

터미널 실험 8-6 실행. Spring 오타 검색

```
curl "http://localhost:8080/search?keyword=갈럭시"
curl "http://localhost:8080/search?keyword=스마크폰"
```

[CAPTURE NEEDED: assets/CH08/terminal/08_spring-fuzzy.png | 터미널 두 번의 curl 실행. 첫 번째 "갈럭시" 응답에 갤럭시 3건. 두 번째 "스마크폰" 응답에 스마트폰 관련 문서 2건. 정상 키워드로 친 8-13 과 거의 같은 결과가 오타에도 나오는 것이 포인트]

그림 8-15. 오타가 정상 검색과 비슷한 결과를 돌려줍니다. 사용자는 검색창에서 한 글자쯤 실수해도 결과를 만납니다

Kibana 와 Spring 결과가 다를 때 디버깅 순서

- **ES 에서만 결과가 나오고 Spring 은 빈 배열**: `application.properties` 의 `spring.elasticsearch.uris` 가 `localhost:9200` 인지 `elasticsearch:9200` 인지 확인합니다 (실행 위치에 따라 다름)
- **ES 쿼리 응답은 있는데 Spring 응답이 비어 있음**: ES 의 `_id` 와 RDB `id` 가 엇갈렸을 가능성. Kibana 에서 삽입한 문서는 Spring 을 거치지 않았으므로 RDB 에 없습니다. 재조회가 공백이 됩니다
- **Fuzzy 가 안 먹을 때**: `@Field(type = FieldType.Keyword)` 로 바꿔 놓진 않았는지, `fuzziness` 를 명시했는지 확인합니다
- **인덱스가 안 만들어짐**: Spring 최초 기동 시 `devices` 인덱스가 자동 생성되는데, ES 가 완전히 기동되기 전에 Spring 이 떠서 실패하면 `docker restart spring-elasticsearch-app` 한 번으로 복구됩니다

8.9 두 저장소를 쓰는 이상 피할 수 없는 문제

오픈이는 숫자를 확인했습니다. 5초가 수십 ms 가 됐고, 띄어쓰기·영문·오타가 모두 풀렸습니다. 수첩에 '해결' 한 줄을 쓰려는데, 팀장이 모니터 쪽에서 한마디를 던졌습니다.

팀장 : "방금 저장한 10건 중에 ES 에 9건만 들어갔다면요."

오픈이는 손이 멈췄습니다.

네트워크가 끊어졌다면. ES 가 잠깐 죽었다면.

현재 코드는 `@Transactional` 안에서 RDB `save()` 가 끝난 뒤 ES `save()` 를 호출합니다. 만약 ES 호출이 예외를 던지면 트랜잭션이 롤백되면서 RDB 저장도 되돌아갑니다. 여기까지는 괜찮아 보입니다. 그런데 문제는 **RDB 가 커밋된 직후 ES 호출이 타임아웃으로 실패하는 경우** 입니다. RDB 는 이미 커밋됐기 때문에 되돌릴 수 없고, ES 에는 해당 문서가 없습니다. 결과적으로 원본과 색인이 **영구적으로 어긋납니다.**

반대 방향도 있습니다. RDB 저장은 성공, ES 저장도 성공했는데, 그 뒤에 **원본을 UPDATE 하는 API** 가 ES 반영을 깜빡 잊고 RDB 만 고치는 경우. 같은 `id` 의 두 저장소가 서로 다른 제목을 들고 있게 됩니다. 사용자는 검색 결과에서는 옛 제목을 보고, 상세 페이지에서는 새 제목을 봅니다.

더 큰 문제는 **응답 지연** 입니다. 지금 구조는 저장 API 한 번에 RDB 왕복 1회 + ES 왕복 1회가 직렬로 붙습니다. ES 가 잠깐 느려지면 저장 API 도 같이 느려집니다. 저장은 원본 기록이 본래 목적인데, 검색 색인 때문에 원본 저장이 끌려가는 꼴입니다.

두 저장소를 쓰기로 한 순간부터 이 문제는 따라다닌다.

오픈이는 수첩 아래쪽에 세 가지 방향을 적었습니다.

```
— 생각 정리 —

1. 동기 보상 (retry)
   · ES 저장 실패 시 재시도 로직을 서비스에 직접
   · 코드가 복잡해지고, 재시도 동안 API 응답이 더 늘어남

2. 배치 동기화 (주기적 재색인)
   · RDB 를 진실의 원본으로 두고, 주기적으로 전체 혹은 변경분을 다시 색인
   · 지연이 김. 즉시 검색이 필요한 요구에는 부적합

3. 메시지 큐로 비동기 분리 (발행·구독)
   · RDB 에 변경이 일어나면 "이벤트" 를 큐에 발행
   · 별도의 구독자(consumer) 가 이 이벤트를 받아 ES 에 반영
   · 저장 API 응답은 RDB 커밋만 끝나면 바로 반환
   · ES 가 잠깐 죽어도 이벤트가 큐에 쌓였다가 복구 후 재처리
```

세 번째 방향이 이 문제에 가장 잘 맞는 모양이었습니다. 저장 쪽은 RDB 한 번만 하고 응답을 바로 돌려주고, ES 동기화는 다른 과정이 따로 돌면서 책임집니다. 파일 변경·문서 수정 같은 이벤트를 큐로 흘려보내는 구조는 검색뿐 아니라 알림·로그·외부 연동 같은 여러 자리에서 같이 쓰입니다.

동료 : "다음 챕터에서 그 큐를 붙여요."

오픈이 : "네. 파일이 바뀌면 알림을 큐에 발행하고, 구독자가 받아서 ES 든 어디든 동기화하는 구조. 이름은 RabbitMQ 예요."

팀장이 모니터 너머로 엄지를 한 번 들어 보였습니다. 오픈이는 수첩을 덮었습니다.

한 테이블을 풀스캔하던 LIKE 쿼리를 카드 한 장으로 바꿨다. 이제 그 카드를 안전하게 관리할 차례다.

다음 챕터에서 RabbitMQ 를 얹어, 파일이 바뀌면 알림이 발행되고 구독자가 자동으로 반영하는 구조로 이 불일치를 풉니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
책 뒤에 붙은 색인 카드	역색인 (Inverted Index)	"단어 → 문서 번호 목록" 으로 뒤집은 검색용 자료구조. 문서 저장 시 토큰별로 문서 `_id` 목록을 기록해 두어, 검색 시 전체 스캔 없이 해당 목록만 읽어 매칭 문서를 찾음
문장을 단어로 쪼개는 사서	분석기 (Analyzer)	텍스트를 토큰으로 쪼개는 규칙. `character filter → tokenizer → token filter` 3단계로 동작. 저장 시점과 검색 시점에 같은 분석기가 적용돼야 일관된 매칭이 이뤄짐
한국어용 사서	`nori` 형태소 분석기	Elasticsearch 에 번들된 한국어 분석기. 조사·어미를 인식해 "연차유급휴가" 를 "연차 / 유급 / 휴가" 로 분해. 기본 표준 분석기로는 한국어 단어 경계를 잡기 어렵기 때문에 별도 플러그인 사용
책 한 권	문서 (Document)	ES 에 저장되는 JSON 단위 레코드. RDB 의 행(row) 과 유사. `_id`, `_source`, `_index` 등의 메타 필드를 가짐

서가 한 구획	인덱스 (Index)	같은 종류의 문서를 모아 둔 논리 단위. RDB 의 테이블과 유사하지만, 내부적으로는 샤드로 분산 저장됨
단어용 칸 vs 코드용 칸	`Text` vs `Keyword` 타입	`Text` 는 분석기를 거쳐 토큰화되는 검색용 필드. `Keyword` 는 분석 없이 문자열 통째로 저장되는 정확 매칭용 필드
ES 클래스 어노테이션	`@Document(indexName = "...")`	자바 클래스를 특정 ES 인덱스 문서로 매핑하는 Spring Data ES 어노테이션. RDB 의 `@Entity` 에 대응
ES 필드 어노테이션	`@Field(type = FieldType.Text)`	필드를 특정 ES 필드 타입으로 매핑. `Text`, `Keyword`, `Integer`, `Date` 등 사용 가능
카드 스캐너	`match` · `multi_match`	분석기를 거쳐 토큰 단위로 매칭하는 쿼리. `multi_match` 는 여러 필드를 동시에 검색하고, 각 필드에 `^N` 으로 가중치 부여 가능
카드 점수	관련도 점수 (`_score`)	Lucene 의 BM25 알고리즘이 계산하는 매칭 점수. 토큰 빈도(TF), 문서 빈도(IDF), 필드 길이, 가중치 (`^N`) 등을 고려해 결정
느슨한 카드 매칭	`bool` + `should` + `minimumShouldMatch`	여러 조건 중 지정 개수 이상이 매칭되면 결과에 포함. `should` 는 OR 에 가깝고, `minimumShouldMatch` 로 최소 매칭 수를 조절
오타 이웃 카드	`fuzziness` (Fuzzy 쿼리)	편집 거리(Levenshtein Distance) 안쪽의 이웃 토큰을 함께 매칭하는 기능. `AUTO` · 정수 · `0/1/2` 단계로 허용 수준 설정
한 글자 수정	편집 거리 (Edit Distance)	한 문자열을 다른 문자열로 변환하는 데 필요한 최소 편집 횟수. 교체·추가·삭제 세 연산의 합

길이별 허용 자동 결정	`fuzziness: AUTO`	토큰 길이 0~2 는 허용 거리 0, 3~5 는 1, 6 이상은 2 로 자동 설정. 짧은 단어의 오탐을 억제
두 저장소 동시 저장	Dual Write	같은 데이터를 두 저장소에 순차·병행으로 저장하는 패턴. 트랜잭션이 둘을 묶지 못하기 때문에 실패 시 보상 처리가 필요
원본 id 재사용	식별자 동기화	ES 문서의 `_id` 로 RDB PK 와 같은 값을 쓰는 설계. 검색 결과의 id 를 그대로 RDB 재조회 키로 사용 가능
id 로 원본 꺼내기	ES → RDB 재조회	ES 에서 매칭된 문서의 `id` 목록만 추출한 뒤, `findAllById` 로 RDB 에서 최신 본문을 가져오는 패턴. 원본 신선도와 검색 성능을 동시에 확보
복잡한 쿼리 조립기	`NativeQuery` + `ElasticsearchOperations`	Spring Data ES 에서 Query DSL 을 자바 메서드 체이닝으로 표현하는 API. `ElasticsearchRepository` 로 표현하기 어려운 `multi_match`, `bool`, `fuzziness` 등에 사용
브라우저 관리 도구	Kibana Dev Tools	ES 클러스터에 직접 REST 쿼리를 보낼 수 있는 관리·디버그 콘솔. 인덱스 조회, 매핑 확인, DSL 테스트 등에 사용

이것만은 기억하자

- **역색인은 "책 뒤 색인 카드"다.** RDB 는 10만 권을 한 장씩 넘기지만, ES 는 단어별 카드 한 장만 보고 문서 번호로 바로 점프합니다. `LIKE '%강의%'` 의 5초가 ms 로 떨어지는 이유는 구조가 달라서입니다
- **저장도 검색도 같은 분석기를 거친다.** `@Field(type = FieldType.Text)` 는 "이 필드를 토큰으로 쪼개 역색인에 넣겠다" 는 선언입니다. 검색어도 같은 분석기로 쪼개지기 때문에 "자바 기초" 와 "기초 자바" 가 같은 두 토큰으로 매칭됩니다. 정확 매칭이 필요한 필드는 `Keyword` 로 따로 둡니다

- **원본은 RDB, 색인은 ES.** 저장은 `@Transactional` 안에서 RDB 먼저, 같은 id 로 ES 를 이어 씁니다. 검색은 ES 에서 `id` 만 받고, 응답 본문은 RDB 에서 `findAllById` 로 가져옵니다. 두 저장소의 역할이 섞이지 않게 합니다
- `multi_match` + `fuzziness("AUTO")` **한 줄이 검색 품질을 바꾼다.** `title^3` 로 제목 가중, `fuzziness("AUTO")` 로 오타 허용, `bool should + minimumShouldMatch("1")` 로 느슨한 매칭. 세 옵션이 키보드 실수·띄어쓰기·영문을 흡수합니다
- **두 저장소를 쓰는 순간 불일치 가능성이 생긴다.** RDB 는 커밋됐는데 ES 저장이 실패하거나, RDB 만 UPDATE 하고 ES 를 깜빡하면 원본과 색인이 어긋납니다. 다음 챕터에서 RabbitMQ 로 변경 이벤트를 발행·구독하는 구조를 얹어 이 문제를 분리해 풉니다